

Shoa Real Estate Development Documentation

Project: Shoa Homes Real Estate PLC

Stack: React (Frontend), Node.js + Express (Backend), PostgreSQL (Database), Cloudinary (Media Storage)

Status: Landing page, backend integration and dashboard features.

1. Overview

Mission:

To provide a modern, digital platform for Shoa Homes Real Estate PLC, empowering clients to discover, explore, and inquire about properties while allowing administrators to manage listings and analytics efficiently.

Core MVP Features:

- Access Control (Admin only)
- Property Management (CRUD)
- Image Uploads (via Cloudinary)
- Search, Filters & Pagination
- Property Detail Pages
- Contact/Inquiry System
- Admin Dashboard with Analytics

Design & Integration:

The project will follow a clean, responsive, image-driven UI consistent with Shoa Homes' branding and color palette. It will ensure seamless performance across devices (desktop, tablet, mobile).

2. UI / UX Designer Deliverables

Goals:

Extend the modern landing page UI into full user workflows for clients and admins, optimized for mobile-first usage.

Tasks:

Wireframes & User Flows:

- Property Listing & Search Flow (filters, pagination, detail view)
- Inquiry Flow (form submission → admin inbox)
- Admin Dashboard (analytics, listing management)

Design Guidelines:

- Consistent color palette and typography
- Clean, photo-focused layouts with strong imagery
- Accessibility (WCAG 2.1 compliance)
- Responsive design for all devices

Final Deliverables:

- High-fidelity Figma mockups for all core pages
- Interactive prototype for stakeholder approval
- Exported assets (icons, SVGs) for development

3. Frontend Developer Deliverables (React + Tailwind)

Tasks:

Component Development:

- **Listing UI:** Property cards (image, price, location, specs)
- **Search & Filters:** Dynamic filter panel and keyword search
- **Property Details:** Image gallery, map integration, inquiry form
- **Admin Dashboard:** Property CRUD forms, inquiry table, analytics charts (Recharts/Chart.js)

API Integration:

- Connect to backend endpoints via Axios or React Query
- Implement JWT-based authentication
- Use Cloudinary for image upload

- Handle pagination, sorting, and error states

Performance & SEO:

- Mobile-first responsiveness
- Lazy-loading for images
- SEO meta tags, OpenGraph, sitemap integration

Final Deliverables:

- Fully responsive frontend (Client + Admin)
- Integrated API calls and authentication
- Cross-browser and device testing completed

4. Backend Developer Deliverables (Node.js + Express + PostgreSQL)

Tasks:

Database Design (Prisma)

API Development

- **Auth API:** JWT-based login/register with bcrypt hashing
- **Property API:** CRUD operations for admins, list/search/filter endpoints for clients
- **Inquiry API:** Submit and manage property inquiries
- **Dashboard API:** Analytics (total listings, inquiries, users, etc.)

File & Media Handling

- Integrate **Cloudinary** for image uploads
- Store URLs and metadata in database
- Support multiple image uploads with secure access

Security & Access

- Admin authentication and JWT for admin dashboard access.
- Input validation & sanitization (protect from XSS/SQL injection)

- HTTPS & environment-based config for secrets
- Rate limiting on inquiry & auth routes

Final Deliverables

- Fully functional REST API (Auth, Properties, Inquiries, Dashboard)
 - PostgreSQL schema with migrations
 - Integrated Cloudinary upload endpoints
 - API documentation (Swagger or Postman Collection)
-

5. Integration & Testing

Frontend ↔ Backend Integration:

- Test all endpoints from React components
- Handle edge cases (invalid input, failed upload, offline state)

Testing Strategy:

- **Unit Tests:** Jest for backend logic, React Testing Library for UI
 - **E2E Tests:** Cypress for property & inquiry workflows
 - **Security Tests:** Verify role-based routes, JWT validation, file upload safety
-

6. Deployment

CI/CD:

- GitHub Actions for test & deploy
- Separate **dev**, **staging**, and **production** environments

Hosting:

- Frontend → Vercel
- Backend → Render / AWS
- Database → Managed PostgreSQL

Environment Variables:

```
DB_URL=  
CLOUDINARY_KEY=  
CLOUDINARY_SECRET=  
JWT_SECRET=  
EMAIL_API_KEY=
```

7. Project Timeline

Week	Deliverables
Week 0–1	UI/UX Design + Auth & Database Setup
Week 2–3	Property CRUD APIs + Frontend Listing & Search + Image Upload + Property Detail + Inquiry System
Week 3–4	Bug Fixes + Final QA + Deployment

7. Additional Payment Requirement – Web Hosting for Backend

No	Hosting Type	Description	Estimated Cost (ETB)
1	Shared Hosting	Basic hosting suitable for small static websites. Limited resources.	7,400 – 8,300 / Year
2	VPS Hosting (Recommended)	Virtual Private Server suitable for Node.js backend and PostgreSQL database.	1,800 – 3,600 / Month